

수업 02 · 전체 과정 6/20일차

Python 기초

2026-10-14 (수) · 10:00~17:00

50분 수업 · 10분 휴식 · 중식 11:50~13:00

짧은 Python 코드를 읽고 실행합니다. 변수, 조건, 반복을 이용해 센서 기록을 점검해 봅니다.

교안 읽기	직접 해보기	결과 확인하기
개념과 완성 예제를 먼저 봅니다.	필수 활동을 열어 작성·수정합니다.	예상과 실제를 비교하고 설명합니다.

처음 배우는 사람을 위한 수업

파일과 폴더, 저장과 실행부터 설명합니다. 설치·계정 문제가 있으면 제공 예제의 수동 추적이나 강사 시연으로 같은 개념을 확인하고, 실제 실행 여부를 구분해 기록합니다.

웹 강의실 열기 · 화면의 링크와 내려받기 자료를 함께 사용하세요.

PDF 제작일 2026-10-05 · 원본 웹 내용 + 50분 교시 운영·설명 보완

이 교안의 순서

오늘 준비할 것	3
오늘의 6교시	6
1교시 · 파일에서 출력까지	7
2교시 · 값에 이름 붙이기	12
3교시 · 문자와 숫자의 연산	20
4교시 · 기준에 따라 나누기	27
5교시 · 여러 기록을 반복하기	34
6교시 · 작은 점검 프로그램	41
추가 활동 · 필수 경로를 마친 뒤	49
용어와 코드 기호 찾아보기	50
오늘 배운 내용 확인하기	51
파일과 다음 수업	53

오늘 준비할 것

Python 기초 · 값과 실행 흐름

2026-10-14 · Python 기초

짧은 Python 코드를 읽고 실행합니다. 변수, 조건, 반복을 이용해 센서 기록을 점검해 봅니다.

원본은 그대로 두고 work 폴더의 지정한 파일을 수정하세요. 계정이나 실제 장비 없이 합성 자료로 진행합니다.

오늘의 연결

지난 데이터 수업에서 읽은 온도와 식별자를 오늘은 Python 코드로 다뤄 봅니다.

먼저 구별할 화면

- 파일 탐색기는 저장한 파일의 위치를 찾습니다.
- 텍스트 편집기는 코드와 데이터를 읽고 고칩니다.
- 터미널은 명령을 입력하고 실제 실행 결과를 확인합니다.
- 웹 교안은 설명·원문·출력 예를 제공합니다. Python 실행기는 아닙니다.

참여 순서

1. 압축을 일반 폴더에 풀고 data-code-work의 위치를 유지합니다.
2. lesson의 개념을 읽고 activity-guide의 해당 활동 입력을 엽니다.
3. work의 답칸에 예상과 이유를 먼저 쓰고 지정한 값을 수정합니다.
4. 저장 후 정확한 work 파일을 실행하거나 강사 출력·제공 예와 수동 비교합니다.
5. 확인 방식을 기록합니다. 개인 실행을 하지 않았으면 실행 성공으로 적지 않습니다.

도구를 준비하지 못했을 때

설치 문제로 수업을 멈추지 말고 제공한 전체 코드·출력과 동일 입력을 사용해 줄별 값을 직접 추적합니다. 종이에도 같은 답칸을 작성합니다. 개인 실행은 강사가 환경을 준비한 뒤 확인합니다.

[교안](#) · [실습](#) · [용어](#)

Windows · 처음 실행할 위치 만들기

제작 가정: Windows 탐색기와 PowerShell을 사용합니다. 다른 OS는 강사의 해당 환경 안내를 따릅니다. 설치 문제는 개념 평가와 따로 기록합니다.

1. 예제 ZIP을 우클릭 → **모두 추출** → **추출**합니다. ZIP 안을 보는 창에서 실행하지 않습니다.

2. 탐색기로 압축을 푼 곳의 **course** 폴더를 엽니다. 안에 **data**, **code**, **work** 세 폴더가 실제로 보이는지 확인합니다. 폴더가 한 겹 더 있으면 그 안으로 들어갑니다.
3. 탐색기 위 주소 표시줄을 클릭해 **powershell**을 입력하고 Enter를 누릅니다. 현재 폴더에서 PowerShell 창이 열립니다. 편집기에서 파일을 연 것만으로 실행 위치가 정해지지는 않습니다.
4. 아래 명령을 한 줄씩 입력하고 Enter를 누릅니다. 앞의 **PS ...>** 문구는 입력하지 않습니다.

```
Get-Location
Get-ChildItem -Directory
Test-Path .\work\A02_print.py
```

Get-Location은 지금 실행할 폴더의 실제 경로를 보여 줍니다. 끝이 **course**인지 확인합니다. **Get-ChildItem** 목록에는 **data**, **code**, **work**가 보여야 합니다. 마지막 명령은 파일이 있는지 확인하며 정상 결과는 **True**입니다. 내 실제 경로: ____ / 확인 결과: ____.

1. **work/A02_print.py** 첫 print 문구를 **오늘의 점검 시작**으로 고치고 저장합니다. 같은 PowerShell 창에서 실행합니다.

```
python -X utf8 work/A02_print.py
```

예상 두 줄은 다음과 같습니다.

```
오늘의 점검 시작
S01 22.5
```

- **False, can't open file, No such file:** 실제 폴더를 탐색기로 다시 열고 2~4단계를 반복합니다. 파일 이름이 **.py.txt**가 아닌지 탐색기의 **보기 → 표시 → 파일 확장명**에서 확인합니다.
- 첫 줄이 **Python 점검 시작:** 수정 파일을 저장했는지, 실행 명령이 **code/**가 아닌 **work/**를 가리키는지 확인합니다.
- **python**을 찾지 못하거나 Store 안내가 뜬: 강사에게 설치 상태를 확인받습니다. 설치를 혼자 진행하지 말고 예상·제공 출력 비교로 합류하고 **개인 실행 미완료**를 적습니다.

실행 위치 확인 → 파일 저장 → 명령 입력 → 실제 출력 기록을 서로 다른 행동으로 확인합니다.

수업 전, 이 세 가지부터 확인하세요

앞선 수업을 기억하지 못해도 아래 질문과 풀이부터 다시 읽으면 됩니다. 먼저 자기 말로 답한 뒤 풀이와 비교하세요.

데이터와 명령

질문: 22.5와 `print(22.5)`는 무엇이 다른가요?

풀이: 22.5는 값입니다. `print(22.5)`는 그 값을 화면에 표시하라는 호출입니다. 저장된 명령을 실행해야 출력이 생깁니다.

따옴표

질문: "22.5" + "1"의 결과가 23.5인가요?

풀이: 문자열을 잇는 결과는 "22.51"입니다. 계산할 숫자인지 식별자인지 먼저 결정하고 자료형을 확인합니다.

분기

질문: 온도가 기준과 정확히 같으면 > 조건은 참인가요?

풀이: 거짓입니다. 경계값을 직접 비교해야 >와 >=의 업무 의미 차이를 이해할 수 있습니다.

오늘의 수업과 복습을 나누는 방법

수업은 10:0010:50, 11:0011:50, 13:0013:50, 14:0014:50, 15:0015:50, 16:0016:50입니다. 중식은 11:5013:00이며 수업 사이에는 10분 쉽니다. 마지막 16:5017:00은 휴식과 저장·퇴실 준비로 배치했습니다.

교시	수업에서 먼저 확인할 활동	추가·복습
1	A01, A02	A03
2	A04, A05, A06	새 입력으로 한 번 더 확인
3	A07, A08, A09	새 입력으로 한 번 더 확인
4	A10, A11, A12	새 입력으로 한 번 더 확인
5	A13, A14, A15	새 입력으로 한 번 더 확인
6	A16, A17, A18	새 입력으로 한 번 더 확인

교안의 쉬운 사례와 단계별 풀이를 먼저 읽고 필수 결과를 직접 만들어 봅니다. 새 독립 문제는 결과를 확인한 뒤 복습하거나 도전합니다. 모든 페이지를 같은 시간에 끝내라는 뜻은 아닙니다. 설치가 막혔을 때는 제공 출력과 수동 추적으로 개념을 확인하고, 직접 실행했는지는 따로 적습니다.

오늘의 6교시

교시·시간	학습 초점	필수 활동
1교시 10:00~10:50	코드 파일과 출력	A01, A02
2교시 11:00~11:50	이름에 값을 연결하기	A04, A05, A06
3교시 13:00~13:50	문자·수와 작은 계산	A07, A08, A09
4교시 14:00~14:50	비교와 조건의 갈래	A10, A11, A12
5교시 15:00~15:50	목록과 반복	A13, A14, A15
6교시 16:00~16:50	작은 점검 프로그램	A16, A17, A18

설명·시연·함께 수행·개별 수행·풀이를 합쳐 교시당 50분입니다. 추가 활동까지 모두 같은 시간에 끝내라는 뜻은 아닙니다. 마지막 10분은 휴식·종료 준비로 배치합니다. 실제 학습자 소요시간은 수업에서 측정해 조정합니다.

2026-10-14 · Python 기초

짧은 Python 코드를 읽고 실행합니다. 변수, 조건, 반복을 이용해 센서 기록을 점검해 봅니다.

확정 운영은 50분×6교시·순수 수업 300분이며 중식은 11:50~13:00입니다. 실제 학습자 리허설 전입니다. 원본을 보존하고 work의 지정한 부분을 수정합니다. 계정이나 실제 장비 없이 합성 자료로 진행합니다.

1교시 · 파일에서 출력까지

10:00~10:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 코드 파일과 출력

먼저 할 활동: A01, A02

추가·복습: A03 · 필수 결과를 확인한 뒤 진행합니다.

A01 · 입력·처리·출력과 코드 파일

사례로 이해하기 · 편집 화면·저장 파일·실행 결과의 세 상태

필수 설명 보충입니다. 강사 시연 → 함께 추적 → 예상과 실제 비교 → 자기 말로 설명 순서로 읽습니다. 아래 독립 문제는 복습·도전이며 기존 50분 필수 경로와 별도로 활용합니다.

접수 담당자가 안내 문구를 ‘오늘 접수’에서 ‘예약 접수’로 고쳤는데 실행 화면에는 여전히 이전 문구가 나옵니다. 컴퓨터가 마음대로 문구를 바꾼 것이 아닐 수 있습니다. 편집기에 보이는 글, 디스크에 저장된 파일, 그 파일을 실행해 얻은 화면은 서로 다른 상태입니다. 종이에 작성 중인 신청서와 제출한 사본과 처리 결과를 구별하는 것처럼 세 상태를 순서대로 확인합니다. 저장 전 편집 내용은 실행 대상 파일에 아직 반영되지 않았을 수 있습니다.

폴더는 파일을 모은 위치이고 경로는 특정 파일까지 가는 주소입니다. course 안의 code는 완성 예제, work는 학생 작업 본으로 사용합니다. 둘 다 notice.py라는 이름이어도 주소가 다르면 다른 파일입니다. .py는 Python 명령을 담은 파일의 확장자입니다. 텍스트 편집기로 내용을 읽는 일은 실행이 아닙니다. 실행은 터미널에서 Python에게 파일 주소를 주고 명령을 처리하도록 요청하는 행동입니다. 명령 뒤 화면에 나온 문구는 출력이며 파일 내용과 역할이 다릅니다.

추적표의 완성 예를 봅니다. work/notice.py를 열어 문구 수정 → 저장 → course 폴더의 터미널에서 work/notice.py를 실행 → 예약 접수 출력 순서입니다. 수정하고 code/notice.py를 실행했다면 원본의 오늘 접수가 나올 수 있습니다. 저장하지 않았다면 work를 실행해도 이전 내용이 나올 수 있습니다. 실행 파일 주소와 저장 여부를 각각 확인하므로 같은 증상이 같은 원인이라고 단정하지 않습니다. 오류가 나면 입력 명령과 파일 이름을 그대로 남기고 파일 확장명이 .py.txt인지도 확인합니다. 개인 실행이 준비되지 않았다면 세 상태 그림과 예상 결과를 종이에 작성합니다.

새 상황에 독립적으로 적용하기

복습·도전: 편집기는 work/check.py에 ‘검사 B’를 보여 줍니다. 실행 명령은 code/check.py를 가리키고 출력은 ‘검사 A’입니다. 확인된 사실 세 가지와 다음 확인 동작 두 가지를 적으세요. 새 설치나 임의 파일 삭제는 필요하지 않습니다.

정답과 오답의 이유

편집 경로·실행 경로가 다르며 화면 출력은 A입니다. work 파일을 저장했는지 확인하고 같은 work 주소를 실행합니다. 파일이 다르다는 증거는 있지만 저장 여부는 아직 미확인입니다. ‘Python 고장’은 증거 없는 원인 단정입니다.

핵심 세 질문과 해설

1. 저장과 실행은 같은 행동인가요? 저장은 파일 내용을 남기고 실행은 그 파일의 명령을 처리합니다.

2. **파일 이름만 같으면 같은 파일인가요?** 폴더까지 포함한 경로가 같아야 같은 대상을 찾습니다.

3. **설치 실패면 개념도 틀린가요?** 환경 준비와 파일·실행 개념의 이해는 따로 확인합니다.

개념. 프로그램은 입력으로 받은 값을 정해진 순서대로 처리하여 결과를 만드는 명령의 묶음입니다. 주문서를 받아 금액을 계산한 뒤 영수증을 쓰는 일과 비슷하지만, 컴퓨터는 작성자가 생략한 의도를 알아서 채우지 않습니다.

왜 필요한가. 코드와 결과를 구별해야 어디를 고쳤고 무엇이 바뀌었는지 설명할 수 있습니다. .py는 Python 코드를 담는 파일의 확장자이며, 파일을 열어 읽는 행동과 실행하는 행동은 다릅니다.

함께 보는 예. 10/8의 온도 22.5와 수업 기준 24.0을 입력이라고 해 봅니다. 두 수를 비교하는 것이 처리이고 '정상'이라는 표시가 출력입니다. 기준은 수업에서 만든 규칙이며 실제 장비 안전 판정이 아닙니다.

헛갈리는 경우. 파일 이름을 result.py로 바꾼다고 결과 파일이 되지 않습니다. 내용을 읽어 명령인지 기록인지 확인해야 합니다. 화면에서 '정상'이라는 글자를 보았어도 어느 파일을 어떤 값으로 실행했는지 모르면 재현하기 어렵습니다.

제공 입력

업무 카드: S01 온도 22.5°C, 기준 24.0°C. 기준보다 크면 '확인', 아니면 '정상'. 파일 카드: code/A02_print.py, work/A02_print.py, 출력창.

A01 · 따라 하기와 작성칸

A01 · 입력·처리·출력과 코드 파일

목적: 코드와 결과를 구별해야 어디를 고쳤고 무엇이 바뀌었는지 설명할 수 있습니다. .py는 Python 코드를 담는 파일의 확장자이며, 파일을 열어 읽는 행동과 실행하는 행동은 다릅니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A01_input.md](#)

- [A01_notes.md](#)

단계

1. 카드에서 입력 두 값과 처리 규칙을 밑줄로 구분합니다.
2. 예상 출력 한 단어와 그렇게 판단한 이유를 씁니다.
3. 원본 코드와 작업 코드의 폴더를 찾아 서로 다른 파일임을 표시합니다.
4. 코드 열기·저장·실행·출력을 자신의 말로 한 문장씩 설명합니다.

예상 결과: 입력 22.5/24.0, 처리 > 비교, 출력 정상. 파일을 여는 것과 실행하는 것을 구별합니다.

확인 방법: 기준을 22.0으로 바꾸면 어느 단계의 값이 바뀌며 출력은 어떻게 될까요?

오류 해결: 입력과 출력이 섞이면 컴퓨터가 일을 하기 전 이미 주어진 값과 일을 한 뒤 생긴 표시를 다른 색으로 적습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
 실제 결과·근거: _____
 다음에 확인할 것: _____

A02 · 첫 출력과 주석

개념. print는 전달받은 값을 화면에 보여 주는 기본 함수입니다. 함수라는 말은 우선 이름 붙은 일을 호출하는 도구로 이해합니다. 괄호 안에 보여 줄 내용을 넣고, 문자열은 따옴표로 감쌉니다.

왜 필요한가. 짧은 출력을 직접 바꾸면 편집한 내용이 실행에 반영되는 과정을 볼 수 있습니다. 복잡한 프로그램을 만들기 전에 실행 통로를 확인하는 작은 성공 기준입니다.

함께 보는 예. 첫 줄의 # 뒤 문장은 사람이 읽는 설명인 주석입니다. Python은 이 주석을 실행하지 않습니다. 따옴표 안의 #는 글자의 일부이므로 주석이 아니라는 점도 비교합니다.

헛갈리는 경우. print라는 이름을 Print로 쓰면 같은 도구가 아닙니다. 큰따옴표는 키보드로 입력하는 곧은 따옴표이며 문서 편집기의 장식 따옴표로 바꾸면 문법 문제가 생길 수 있습니다.

전체 코드

```
# 화면에 수업 시작을 표시합니다.
print("Python 점검 시작")
print("S01", 22.5)
```

각 줄의 뜻

줄	코드	뜻
1	# 화면에 수업 시작을 표시합니다.	사람을 위한 설명이며 실행 결과를 만들지 않습니다.
2	print("Python 점검 시작")	문자열 한 개를 화면에 보여 줍니다.
3	print("S01", 22.5)	문자열과 숫자를 쉼표로 나누어 한 줄에 출력합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A02_print.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A02_print.py
```

원본 코드의 예상 출력:

```
Python 점검 시작
S01 22.5
```

작은 수정: Python 점검 시작을 **오늘의 점검 시작**로 바꿉니다.

수정 후 예상 출력:

```
오늘의 점검 시작
S01 22.5
```

Windows · 처음 실행할 위치 만들기

제작 가정: Windows 탐색기와 PowerShell을 사용합니다. 다른 OS는 강사의 해당 환경 안내를 따릅니다. 설치 문제는 개
념 평가와 따로 기록합니다.

1. 예제 ZIP을 우클릭 → **모두 추출** → **추출**합니다. ZIP 안을 보는 창에서 실행하지 않습니다.
2. 탐색기로 압축을 푼 곳의 **course** 폴더를 엽니다. 안에 **data, code, work** 세 폴더가 실제로 보이는지 확인합니다. 폴더
가 한 겹 더 있으면 그 안으로 들어갑니다.
3. 탐색기 위 주소 표시줄을 클릭해 **powershell**을 입력하고 Enter를 누릅니다. 현재 폴더에서 PowerShell 창이 열립니다.
편집기에서 파일을 연 것만으로 실행 위치가 정해지지는 않습니다.
4. 아래 명령을 한 줄씩 입력하고 Enter를 누릅니다. 앞의 **PS ...>** 문구는 입력하지 않습니다.

```
Get-Location
Get-ChildItem -Directory
Test-Path .\work\A02_print.py
```

Get-Location은 지금 실행할 폴더의 실제 경로를 보여 줍니다. 끝이 **course**인지 확인합니다. **Get-ChildItem** 목록에는
data, code, work가 보여야 합니다. 마지막 명령은 파일이 있는지 확인하며 정상 결과는 **True**입니다. 내 실제 경로: ____
/ 확인 결과: ____.

1. **work/A02_print.py** 첫 print 문구를 **오늘의 점검 시작**으로 고치고 저장합니다. 같은 PowerShell 창에서 실행합니다.

```
python -X utf8 work/A02_print.py
```

예상 두 줄은 다음과 같습니다.

```
오늘의 점검 시작
S01 22.5
```

- **False, can't open file, No such file:** 실제 폴더를 탐색기로 다시 열고 2~4단계를 반복합니다. 파일 이름이 **.py.txt**가
아닌지 탐색기의 **보기** → **표시** → **파일 확장명**에서 확인합니다.

- 첫 줄이 **Python 점검 시작**: 수정 파일을 저장했는지, 실행 명령이 **code/**가 아닌 **work/**를 가리키는지 확인합니다.

- **python**을 찾지 못하거나 Store 안내가 뜬: 강사에게 설치 상태를 확인받습니다. 설치를 혼자 진행하지 말고 예상·제공 출
력 비교로 합류하고 **개인 실행 미완료**를 적습니다.

실행 위치 확인 → 파일 저장 → 명령 입력 → 실제 출력 기록을 서로 다른 행동으로 확인합니다.

A02 · 따라 하기와 작성칸

A02 · 첫 출력과 주석

목적: 짧은 출력을 직접 바꾸면 편집한 내용이 실행에 반영되는 과정을 볼 수 있습니다. 복잡한 프로그램을 만들기 전에 실행 통로를 확인하는 작은 성공 기준입니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A02_input.md](#)

- [A02_print.py](#)

- [A02_notes.md](#)

- [A02_print.py](#)

단계

1. work/A02_print.py를 텍스트 편집기로 엽니다.
2. 수정 전 출력 두 줄을 예상해 노트에 씁니다.
3. 첫 안내 문구를 '오늘의 점검 시작'으로 바꾸고 저장합니다.
4. course 폴더의 터미널에서 python -X utf8 work/A02_print.py를 실행하고 예상과 대조합니다.

예상 결과: 첫 안내 문구만 바뀌며 둘째 줄 S01 22.5는 유지됩니다.

확인 방법: 주석의 문구만 바꾸면 출력은 달라질까요?

오류 해결: python을 찾지 못하면 설치를 혼자 진행하지 말고 강사에게 환경을 확인합니다. 그동안 제공된 출력과 수정안을 비교하고 개인 실행 미완료를 표시합니다.

Windows · 처음 실행할 위치 만들기

제작 가정: Windows 탐색기와 PowerShell을 사용합니다. 다른 OS는 강사의 해당 환경 안내를 따릅니다. 설치 문제는 개 념 평가와 따로 기록합니다.

1. 예제 ZIP을 우클릭 → **모두 추출** → **추출**합니다. ZIP 안을 보는 창에서 실행하지 않습니다.
2. 탐색기로 압축을 푼 곳의 **course** 폴더를 엽니다. 안에 **data**, **code**, **work** 세 폴더가 실제로 보이는지 확인합니다. 폴더 가 한 겹 더 있으면 그 안으로 들어갑니다.
3. 탐색기 위 주소 표시줄을 클릭해 **powershell**을 입력하고 Enter를 누릅니다. 현재 폴더에서 PowerShell 창이 열립니다. 편집기에서 파일을 연 것만으로 실행 위치가 정해지지는 않습니다.
4. 아래 명령을 한 줄씩 입력하고 Enter를 누릅니다. 앞의 **PS ...>** 문구는 입력하지 않습니다.

```
Get-Location

Get-ChildItem -Directory

Test-Path .\work\A02_print.py
```

Get-Location은 지금 실행할 폴더의 실제 경로를 보여 줍니다. 끝이 **course**인지 확인합니다. Get-ChildItem 목록에는 **data**, **code**, **work**가 보여야 합니다. 마지막 명령은 파일이 있는지 확인하며 정상 결과는 **True**입니다. 내 실제 경로: ____ / 확인 결과: ____.

1. **work/A02_print.py** 첫 print 문구를 **오늘의 점검 시작**으로 고치고 저장합니다. 같은 PowerShell 창에서 실행합니다.

```
python -X utf8 work/A02_print.py
```

예상 두 줄은 다음과 같습니다.

```
오늘의 점검 시작
S01 22.5
```

- **False, can't open file, No such file:** 실제 폴더를 탐색기로 다시 열고 2~4단계를 반복합니다. 파일 이름이 **.py.txt**가 아닌지 탐색기의 **보기 → 표시 → 파일 확장명**에서 확인합니다.

- 첫 줄이 **Python 점검 시작:** 수정 파일을 저장했는지, 실행 명령이 **code/**가 아닌 **work/**를 가리키는지 확인합니다.

- **python**을 찾지 못하거나 Store 안내가 뜬: 강사에게 설치 상태를 확인받습니다. 설치를 혼자 진행하지 말고 예상·제공 출력 비교로 합류하고 **개인 실행 미완료**를 적습니다.

실행 위치 확인 → 파일 저장 → 명령 입력 → 실제 출력 기록을 서로 다른 행동으로 확인합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
 실제 결과·근거: _____
 다음에 확인할 것: _____

2교시 · 값에 이름 붙이기

11:00~11:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 이름에 값을 연결하기

먼저 할 활동: A04, A05, A06

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A04 · 변수는 값에 붙인 이름

사례로 이해하기 · 재고 갱신을 한 줄씩 추적하기

필수 설명 보충입니다. 강사 시연 → 함께 추적 → 예상과 실제 비교 → 자기 말로 설명 순서로 읽습니다. 아래 독립 문제는 복습·도전이며 기존 50분 필수 경로와 별도로 활용합니다.

창고에서 새 물품이 들어오고 일부가 나갑니다. 담당자는 ‘처음 8개, 입고 3개, 출고 2개’를 순서대로 기록합니다. Python에서는 현재 재고에 stock이라는 이름을 붙이고 각 작업이 끝날 때의 값을 연결합니다. 변수는 여러 값을 자동으로 보관하는 이력표가 아닙니다. 같은 이름에 새 값을 대입하면 그 시점의 값이 바뀝니다. 과거 기록이 필요하다면 별도의 기록을 남겨야 합니다. 먼저 종이에 실행 줄·현재 값·출력 세 열을 만듭니다.

이 사례의 완결 코드는 다음과 같습니다. work 폴더에 새 파일 E02_stock.py를 UTF-8로 저장하고 course 폴더의 터미널에서 `python -X utf8 work/E02_stock.py`를 실행합니다. 기존 파일을 덮어쓰지 않습니다.

```
stock = 8
stock = stock + 3
print(stock)
stock = stock - 2
print(stock)
```

첫 줄은 stock에 정수 8을 연결합니다. 둘째 줄은 오른쪽의 현재 stock 8에 3을 더해 11을 만든 다음 왼쪽 이름에 연결합니다. 셋째 줄은 그때의 11을 화면에 표시합니다. 넷째 줄은 현재 11에서 2를 빼 9로 갱신합니다. 마지막 줄은 9를 출력합니다. 따라서 예상 출력은 첫 줄 11, 둘째 줄 9입니다. =는 수학의 양쪽이 같다는 설명이 아니라 오른쪽 계산 결과를 왼쪽 이름에 연결하는 대입 기호입니다.

입고와 출고를 모두 처리한 뒤 출력 두 줄을 적는 것이 아니라 print를 만나는 순간의 값을 기록합니다. 뒤의 갱신이 이미 나온 첫 출력을 고치지는 않습니다. NameError이면 stock 철자와 대소문자, 처음 대입한 줄을 확인합니다. can't open file 이면 작업 파일 저장 위치를 먼저 확인합니다. 실행 환경이 없으면 같은 추적표를 쓰고 강사 실행과 비교했다는 표시를 남깁니다.

새 상황에 독립적으로 적용하기

복습·도전: 물 12병을 준비하고 4병을 배부한 뒤 현재 수량을 안내합니다. 다시 6병을 받아 최종 수량을 안내합니다. 두 안내 값과 세 번의 대입 뒤 현재 수량을 표로 작성하고, 두 print를 어디에 놓을지 판단하세요.

정답과 오답의 이유

대입 뒤 현재 값은 12→8→14이고 안내값은 8과 14입니다. 배부 직후와 추가 입고 직후에 출력해야 합니다. 모든 대입 뒤에 같은 이름을 두 번 출력하면 14·14가 되어 중간 안내를 잃습니다.

핵심 세 질문과 해설

1. **오른쪽 stock은 언제 읽나요?** 그 줄을 실행하는 시점에 연결된 현재 값을 먼저 읽습니다.
2. **=는 ==와 같은가요?** =는 대입이고 ==는 같은지 비교하는 기호입니다.
3. **나중 갱신이 이전 화면을 바꾸나요?** 이미 출력한 결과는 그때의 값으로 남습니다.

개념. 변수는 프로그램이 값을 다시 찾을 수 있도록 붙인 이름입니다. `location = "입구"`는 왼쪽 이름에 오른쪽 값을 연결합니다. 수학에서 등식의 양변이 같다는 주장과 Python의 대입 동작을 구별합니다.

왜 필요한가. 값을 여러 번 사용할 때 이름을 붙이면 목적이 드러나고 한 곳에서 바꾸기 쉽습니다. 10/8의 표에서 `location`과 `temperature_c`라는 항목을 읽었던 경험을 코드 안의 이름으로 연결합니다.

함께 보는 예. `location`과 `temperature_c`를 출력하면 이름 자체의 글자가 아니라 그 이름이 가리키는 값이 나옵니다. `print("location")`처럼 따옴표를 붙이면 이름을 찾지 않고 글자 `location`을 보여 줍니다.

헷갈리는 경우. 한 이름이 무엇을 뜻하는지는 작성자가 정한 약속입니다. 온도를 `x`로 저장해도 실행될 수 있지만 업무 목적을 읽기 어렵습니다. 의미가 분명한 이름을 쓰되 이름만 보고 단위가 맞다고 단정하지 않습니다.

전체 코드

```
location = "입구"
temperature_c = 22.5
print(location)
print(temperature_c)
print("location")
```

각 줄의 뜻

줄	코드	뜻
1	<code>location = "입구"</code>	장소 문자열을 <code>location</code> 이라는 이름에 연결합니다.
2	<code>temperature_c = 22.5</code>	숫자 22.5를 <code>temperature_c</code> 라는 이름에 연결합니다.
3	<code>print(location)</code>	이름이 가리키는 장소 값을 보여 줍니다.
4	<code>print(temperature_c)</code>	이름이 가리키는 온도 값을 보여 줍니다.
5	<code>print("location")</code>	따옴표 안의 글자 <code>location</code> 을 그대로 보여 줍니다.

실행과 예상 출력

현재 위치가 압축을 푼 `course` 폴더일 때 실행합니다.

```
python -X utf8 code/A04_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A04_example.py
```

원본 코드의 예상 출력:

```
입구
22.5
location
```

작은 수정: **입구**를 **창고**로 바꿉니다.

수정 후 예상 출력:

```
창고
22.5
location
```

A04 · 따라 하기와 작성칸

A04 · 변수는 값에 붙인 이름

목적: 값을 여러 번 사용할 때 이름을 붙이면 목적이 드러나고 한 곳에서 바꾸기 쉽습니다. 10/8의 표에서 location과 temperature_c라는 항목을 읽었던 경험을 코드 안의 이름으로 연결합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A04_input.md](#)
- [A04_example.py](#)
- [A04_notes.md](#)
- [A04_example.py](#)

단계

1. 값과 이름을 서로 다른 색으로 표시합니다.
2. location을 창고로 바꾸면 변할 출력 줄을 먼저 고릅니다.
3. 작업본에서 문자열 한 곳만 바꾸고 저장·실행합니다.
4. 변하지 않은 두 출력 줄의 이유를 씁니다.

예상 결과: 첫 출력만 창고로 바뀌고 온도와 글자 location은 유지됩니다.

확인 방법: print(location)과 print("location")은 왜 다를까요?

오류 해결: NameError가 나오면 대문자·소문자와 밑줄을 비교하고 그 이름에 값을 연결한 줄이 먼저 실행되는지 확인합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
 실제 결과·근거: _____
 다음에 확인할 것: _____

A05 · 다시 대입하면 현재 값이 바뀐다

개념. 코드는 위에서 아래로 실행됩니다. 같은 이름에 새 값을 대입하면 이후에 그 이름을 읽을 때 새 값을 사용합니다. 이전 출력은 이미 화면에 남은 결과이며 새 대입이 과거 화면을 다시 계산하지는 않습니다.

왜 필요한가. 업무 기록을 갱신하는 코드의 순서를 이해하려면 어느 시점의 값을 보고 있는지 추적해야 합니다. 이름은 같은데 출력이 다른 상황을 오류로 오해하지 않도록 한 줄씩 현재 값을 적습니다.

함께 보는 예. 아래 코드에서 첫 출력은 22.5이고, 새 대입 다음 출력은 24.0입니다. 이 순서를 작업 전·작업 후 카드로 나누면 값 변화가 보입니다.

헷갈리는 경우. `temperature_c = temperature_c + 1`은 양변이 같다는 수학식이 아닙니다. 기존 값을 읽고 1을 더한 결과를 같은 이름에 다시 연결하는 명령입니다. 이 활동에서는 단순 재대입부터 확인합니다.

전체 코드

```
temperature_c = 22.5
print(temperature_c)
temperature_c = 24.0
print(temperature_c)
```

각 줄의 뜻

줄	코드	뜻
1	<code>temperature_c = 22.5</code>	처음 온도를 22.5로 정합니다.
2	<code>print(temperature_c)</code>	현재 값을 출력합니다.
3	<code>temperature_c = 24.0</code>	같은 이름에 새 값 24.0을 연결합니다.
4	<code>print(temperature_c)</code>	새로 연결된 현재 값을 출력합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A05_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A05_example.py
```

원본 코드의 예상 출력:


```
22.5
24.0
```

작은 수정: 24.0를 25.0로 바꿉니다.

수정 후 예상 출력:

```
22.5
25.0
```

A05 · 따라 하기와 작성칸

A05 · 다시 대입하면 현재 값이 바뀐다

목적: 업무 기록을 갱신하는 코드의 순서를 이해하려면 어느 시점의 값을 보고 있는지 추적해야 합니다. 이름은 같은데 출력 이 다른 상황을 오류로 오해하지 않도록 한 줄씩 현재 값을 적습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드· 출력으로 수동 추적합니다.

- [A05_input.md](#)
- [A05_example.py](#)
- [A05_notes.md](#)
- [A05_example.py](#)

단계

1. 각 줄 실행 뒤 현재 temperature_c 값을 표에 적습니다.
2. 두 번째 대입값을 25.0으로 바꿀 때 출력 두 줄을 예상합니다.
3. 작업본을 수정해 실행하거나 강사 실행과 대조합니다.
4. 첫 출력이 그대로인 이유를 자신의 말로 씁니다.

예상 결과: 첫 출력 22.5, 둘째 출력 25.0.

확인 방법: 마지막 print를 첫 대입 앞에 옮기면 어떤 문제가 생길까요?

오류 해결: 값이 예상과 다르면 실행 순서대로 현재 값을 다시 써 봅니다. 편집기의 위치와 실제 실행 순서를 건너뛰지 않습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
 실제 결과·근거: _____
 다음에 확인할 것: _____

A06 · 이름 오류를 근거로 고치기

개념. 오류를 고치는 일은 프로그램을 무작정 바꾸는 것이 아니라 보이는 단서로 원인을 좁히는 작업입니다. `NameError`는 실행 중 찾으려는 이름을 알 수 없다는 뜻입니다.

왜 필요한가. 변수의 철자가 다르면 컴퓨터는 서로 다른 이름으로 취급합니다. 업무 의미가 같아 보인다는 이유로 `temp`와 `temperature_c`를 자동 연결하지 않습니다. 작성한 이름과 사용한 이름의 일치를 확인해야 합니다.

함께 보는 예. 아래 정상 예는 `sensor_id`와 `temperature_c`를 각각 정의한 뒤 출력합니다. 고의 오류 카드에는 `print(temperature)`가 있으며, 정의된 이름은 `temperature_c`뿐입니다.

헛갈리는 경우. 오류를 없애려고 새 변수 `temperature = 0`을 무조건 추가하면 원래 온도가 아닌 0이 출력될 수 있습니다. 의도한 값이 어떤 이름에 들어 있는지 먼저 찾아야 합니다.

전체 코드

```
sensor_id = "S01"
temperature_c = 22.5
print(sensor_id, temperature_c)
```

각 줄의 뜻

줄	코드	뜻
1	<code>sensor_id = "S01"</code>	센서를 구별할 문자열을 정합니다.
2	<code>temperature_c = 22.5</code>	섭씨 온도 값을 정합니다.
3	<code>print(sensor_id, temperature_c)</code>	두 이름이 가리키는 값을 함께 출력합니다.

실행과 예상 출력

현재 위치가 압축을 푼 `course` 폴더일 때 실행합니다.

```
python -X utf8 code/A06_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A06_example.py
```

원본 코드의 예상 출력:

```
S01 22.5
```

A06 · 따라 하기와 작성칸

A06 · 이름 오류를 근거로 고치기

목적: 변수의 철자가 다르면 컴퓨터는 서로 다른 이름으로 취급합니다. 업무 의미가 같아 보인다는 이유로 temp와 temperature_c를 자동 연결하지 않습니다. 작성한 이름과 사용한 이름의 일치를 확인해야 합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A06_input.md](#)

- [A06_example.py](#)

- [A06_notes.md](#)

- [A06_example.py](#)

단계

1. data/A06_input.md의 오류 코드에서 정의한 이름과 사용하는 이름을 표시합니다.
2. 원래 뜻을 유지하는 수정 한 곳을 정합니다.
3. work의 정상 예와 비교하며 수정안을 적고 실행 결과를 확인합니다.
4. 원인·수정 위치·다시 확인한 출력을 노트에 기록합니다.

예상 결과: temperature를 temperature_c로 맞추면 S01 22.5가 출력됩니다.

확인 방법: 오류가 사라졌다는 것만으로 올바른 수정이라고 할 수 있나요?

오류 해결: 철자 차이를 찾지 못하면 이름을 한 글자씩 비교합니다. 영문 대소문자와 밑줄도 이름의 일부입니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

3교시 · 문자와 숫자의 연산

13:00~13:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 문자·수와 작은 계산

먼저 할 활동: A07, A08, A09

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A07 · 숫자 더하기와 문자열 연결

사례로 이해하기 · 운송비 계산에서 문자열과 숫자 나누기

필수 설명 보충입니다. 강사 시연 → 함께 추적 → 예상과 실제 비교 → 자기 말로 설명 순서로 읽습니다. 아래 독립 문제는 복습·도전이며 기존 50분 필수 경로와 별도로 활용합니다.

택배 접수표에는 상품번호 "0012", 무게 "2.5", 포장비 1000이 있습니다. CSV에서 읽은 무게 칸은 숫자처럼 보여도 문자열일 수 있습니다. 문자열은 글자 순서를 보존하고 숫자는 수량을 계산합니다. 따옴표로 감싼 "2.5"와 실수 2.5의 저장 종류가 다르므로 계산 전 의미에 맞는 변환을 해야 합니다. 상품번호는 계산용 수량이 아니므로 앞의 0을 보존한 문자열을 유지합니다. 같은 모양의 숫자라도 변환 여부는 업무 목적에 따라 결정합니다.

수업용 요금은 무게 1kg당 2000원에 포장비 1000원을 더하는 것으로 정합니다. 반올림·최소요금 등 실제 운송 규칙은 이번 합성 예에 포함하지 않습니다. "2.5"를 실수 2.5로 읽으면 $2.5 \times 2000 + 1000 = 6000$ 원입니다. 먼저 곱셈으로 무게 요금을 계산하고 포장비를 더합니다. 문자열 "2"와 "5"를 +로 붙이면 "25"가 되어 무게 7이 아닙니다. 문자열을 이어 붙이는 연결과 수치 덧셈을 같은 기호가 다른 종류에 적용된 사례로 비교합니다.

값 추적표는 원문 "2.5"·저장 종류 문자열 → 변환 값 2.5·실수 → 계산값 6000.0·실수 → 업무 표시 6000원 순서입니다. 계산 결과의 .0은 이 예의 저장·표시 방식이지 비용이 소수 원이라는 업무 확정은 아닙니다. 무게 원문이 "미확인"이라면 임의의 0kg로 바꾸지 않고 숫자 변환 전에 확인을 요청합니다. 문자열과 숫자를 +로 바로 섞으면 TypeError가 날 수 있고 "2.5"를 정수로 바로 읽으려 하면 ValueError가 납니다. 종류를 확인한 뒤 실수 변환을 고르고 상품번호까지 함께 바꾸지 않습니다.

새 상황에 독립적으로 적용하기

복습·도전: 판매표의 주문번호 "0009", 단가 "1500", 수량 "3"을 보고 총액과 각 항목의 저장 계획을 쓰세요. 단가는 정수 원, 수량은 개수라는 계약입니다. 배송비 2000원을 포함한 최종액도 계산하세요.

정답과 오답의 이유

주문번호는 문자열 유지, 단가와 수량은 정수로 해석합니다. 상품 총액 $1500 \times 3 = 4500$ 원, 최종액 6500원입니다. "1500"+"3"의 문자열 연결은 "15003"으로 금액 계산이 아닙니다. 주문번호를 정수 9로 바꾸면 앞 0을 잃습니다.

핵심 세 질문과 해설

1. 숫자처럼 보이면 모두 변환하나요? 식별자·등급인지 실제 계산량인지 의미를 먼저 봅니다.

2. 소수 문자열을 정수로 바로 읽나요? 소수가 있는 표현에는 float 같은 의미에 맞는 변환이 필요합니다.
3. 정상 종료면 요금도 맞나요? 단위·요금 규칙·입력 해석이 맞는지 별도 확인해야 합니다.
- 개념.** 숫자의 +는 수치 덧셈을 하고 문자열의 +는 글자를 이어 붙입니다. 같은 모양의 3이라도 따옴표 유무에 따라 저장 자료형이 달라지며 연산의 결과도 달라집니다.
- 왜 필요한가.** CSV의 셀을 기본 방식으로 읽으면 문자열이 된다는 10/8의 내용을 떠올립니다. 수량을 계산하려고 글자를 그대로 이어 붙이면 합계처럼 보이는 잘못된 결과가 생길 수 있습니다.
- 함께 보는 예.** 3 + 2는 5이고 "3" + "2"는 글자 32입니다. 10/8의 사번 001처럼 앞의 0을 유지해야 하는 이름표는 문자열이 적절할 수 있으므로 모든 숫자 모양을 변환하지 않습니다.
- 헛갈리는 경우.** 문자열을 무조건 숫자로 바꾸는 것도 정답이 아닙니다. 사번을 숫자로 바꾸면 식별 표현을 잃을 수 있습니다. 먼저 의미상 수량인지, 이름표인지 판단합니다.

전체 코드

```
count = 3
text_count = "3"
print(count + 2)
print(text_count + "2")
print(type(count).__name__, type(text_count).__name__)
```

각 줄의 뜻

줄	코드	뜻
1	count = 3	숫자 3을 저장합니다.
2	text_count = "3"	글자 3을 저장합니다.
3	print(count + 2)	숫자끼리 덧셈합니다.
4	print(text_count + "2")	문자열끼리 이어 붙입니다.
5	print(type(count).__name__, type(text_count).__name__)	두 값의 자료형 이름을 확인합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A07_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A07_example.py
```

원본 코드의 예상 출력:

```
5
32
int str
```

A07 · 따라 하기와 작성칸

A07 · 숫자 더하기와 문자열 연결

목적: CSV의 셀을 기본 방식으로 읽으면 문자열이 된다는 10/8의 내용을 떠올립니다. 수량을 계산하려고 글자를 그대로 이어 붙이면 합계처럼 보이는 잘못된 결과가 생길 수 있습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A07_input.md](#)
- [A07_example.py](#)
- [A07_notes.md](#)
- [A07_example.py](#)

단계

1. 세 출력 줄을 예상하고 각각 계산인지 글자 연결인지 적습니다.
2. 숫자와 문자열의 +를 같은 기준으로 읽지 않았는지 비교합니다.
3. 고의 오류 text_count + 2에서 서로 다른 자료형을 표시합니다.
4. 수량과 사번 중 무엇을 숫자로 바꿀지 이유를 씁니다.

예상 결과: 5와 문자열 32, int와 str을 구별합니다. 혼합 +는 TypeError가 기대 결과입니다.

확인 방법: "001"에 1을 더해 다음 직원을 정하는 것이 항상 적절할까요?

오류 해결: 화면에서 따옴표 없이 32가 보이면 type 출력도 함께 확인합니다. 표시 모양만으로 자료형을 판단하지 않습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
 실제 결과·근거: _____
 다음에 확인할 것: _____

A08 · 수치 문자열을 숫자로 변환하기

개념. 형 변환은 정해진 규칙으로 다른 자료형의 값을 만드는 일입니다. float("22.5")는 숫자를 나타내는 문자열을 실수값으로 해석합니다. 원문 문자열이 저절로 바뀌는 것은 아니며 결과를 다른 이름에 연결할 수 있습니다.

왜 필요한가. 외부에서 읽은 온도 글자를 계산하려면 숫자로 다룰 값인지 확인하고 변환해야 합니다. 데이터의 의미가 수치라는 사실과 현재 저장 형태가 문자열이라는 사실을 연결하는 단계입니다.

함께 보는 예. raw와 temperature_c를 나누어 두면 받은 원문과 해석 결과를 비교하기 쉽습니다. 출력에서는 두 값의 모양이 같아도 자료형이 str과 float로 다릅니다.

헛갈리는 경우. float("뜨거움")은 수치로 해석할 수 없어 ValueError가 납니다. 빈칸을 0으로 바꾸는 것으로 해결하면 미측정을 실제 0°C로 바꾸게 됩니다. 값 없음 처리는 다음 활동과 다음 날에 별도로 배웁니다.

전체 코드

```
raw = "22.5"
temperature_c = float(raw)
print(raw, type(raw).__name__)
print(temperature_c, type(temperature_c).__name__)
print(temperature_c + 1)
```

각 줄의 뜻

줄	코드	뜻
1	<code>raw = "22.5"</code>	받은 원문 글자를 저장합니다.
2	<code>temperature_c = float(raw)</code>	숫자로 해석한 결과를 새 이름에 연결합니다.
3	<code>print(raw, type(raw).__name__)</code>	원문의 값과 문자열 자료형을 확인합니다.
4	<code>print(temperature_c, type(temperature_c).__name__)</code>	변환 결과와 실수 자료형을 확인합니다.
5	<code>print(temperature_c + 1)</code>	변환한 숫자에 1을 더합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A08_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A08_example.py
```

원본 코드의 예상 출력:

```
22.5 str
22.5 float
23.5
```

작은 수정: "22.5"를 "24.0"로 바꿉니다.

수정 후 예상 출력:

```
24.0 str
24.0 float
25.0
```

A08 · 따라 하기와 작성칸

A08 · 수치 문자열을 숫자로 변환하기

목적: 외부에서 읽은 온도 글자를 계산하려면 숫자로 다룰 값인지 확인하고 변환해야 합니다. 데이터의 의미가 수치라는 사실과 현재 저장 형태가 문자열이라는 사실을 연결하는 단계입니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A08_input.md](#)
- [A08_example.py](#)
- [A08_notes.md](#)
- [A08_example.py](#)

단계

1. 원문과 변환 결과의 이름을 구별합니다.
2. 원문을 "24.0"으로 바꾸고 출력 세 줄을 예상합니다.
3. 저장·실행 뒤 값과 자료형을 대조합니다.
4. 고의 오류 "뜨거움" 사례의 원문·오류 종류·확인 질문을 기록합니다.

예상 결과: 변환 후 계산 결과는 25.0. 숫자가 아닌 글자에서는 ValueError.

확인 방법: 원문 raw를 남기는 이유는 무엇일까요?

오류 해결: ValueError가 보이면 입력 문자열을 먼저 확인합니다. °C 같은 단위 글자가 붙었는지, 빈칸인지 구별하고 확인되지 않은 값을 만들어 채우지 않습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
 실제 결과·근거: _____
 다음에 확인할 것: _____

A09 · 연산과 괄호로 작은 계산 만들기

개념. 연산자는 값으로 계산하는 기호입니다. +, -, *, /를 읽고 괄호로 먼저 계산할 부분을 정합니다. Python에서 /의 결과는 나누어떨어져도 실수 형태가 될 수 있습니다.

왜 필요한가. 합계와 개수를 따로 확인해야 평균의 분모를 잘못 고르는 일을 막을 수 있습니다. 오늘은 이미 확인된 숫자 세 개만 계산하고, 미측정과 잘못된 값이 있는 파일 처리는 다음 날로 넘깁니다.

함께 보는 예. 22.5, 24.0, 23.5의 합은 70.0이며 세 값의 평균은 23.333...입니다. round는 표시를 짧게 하는 데 쓰지만 원래 센서가 정확히 두 자리까지 측정했다는 뜻은 아닙니다.

헛갈리는 경우. 22.5 + 24.0 + 23.5 / 3은 전체 합을 3으로 나누는 식과 다릅니다. 괄호나 합계 이름을 사용해 의도한 계산 순서를 드러냅니다.

전체 코드

```
total = 22.5 + 24.0 + 23.5
count = 3
average = total / count
print(total)
print(round(average, 2))
```

각 줄의 뜻

줄	코드	뜻
1	total = 22.5 + 24.0 + 23.5	확인된 온도 세 값의 합을 계산합니다.
2	count = 3	계산에 사용한 값의 개수를 저장합니다.
3	average = total / count	합을 개수로 나눕니다.
4	print(total)	합계를 출력합니다.
5	print(round(average, 2))	평균을 소수 둘째 자리까지 반올림해 출력합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A09_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A09_example.py
```

원본 코드의 예상 출력:

```
70.0
23.33
```

작은 수정: `count = 3`를 `count = 2`로 바꿉니다.

수정 후 예상 출력:

```
70.0
35.0
```

A09 · 따라 하기와 작성칸

A09 · 연산과 괄호로 작은 계산 만들기

목적: 합계와 개수를 따로 확인해야 평균의 분모를 잘못 고르는 일을 막을 수 있습니다. 오늘은 이미 확인된 숫자 세 개만 계산하고, 미측정과 잘못된 값이 있는 파일 처리는 다음 날로 넘깁니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A09_input.md](#)
- [A09_example.py](#)
- [A09_notes.md](#)
- [A09_example.py](#)

단계

1. 합계와 개수를 손으로 확인합니다.
2. 코드 각 계산의 입력과 결과를 표로 적습니다.
3. `count`를 2로 바꾼 출력 35.0이 계산은 되지만 의미상 맞는지 판단합니다.
4. 정상 분모를 복원하고 0으로 나눌 때 오류 단서를 비교합니다.

예상 결과: 정상 합계 70.0, 평균 23.33. 잘못된 분모는 실행 성공과 별개로 검수해야 합니다.

확인 방법: 코드가 종료 코드 0으로 끝났다면 평균의 업무 의미도 보장되나요?

오류 해결: `ZeroDivisionError`가 생기면 분모가 왜 0인지 먼저 확인합니다. 임의로 1을 넣어 원인을 숨기지 않습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
 실제 결과·근거: _____
 다음에 확인할 것: _____

4교시 · 기준에 따라 나누기

14:00~14:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 비교와 조건의 갈래

먼저 할 활동: A10, A11, A12

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A10 · 비교 결과는 참 또는 거짓

사례로 이해하기 · 이상과 초과를 업무 문장으로 읽기

필수 설명 보충입니다. 강사 시연 → 함께 추적 → 예상과 실제 비교 → 자기 말로 설명 순서로 읽습니다. 아래 독립 문제는 복습·도전이며 기존 50분 필수 경로와 별도로 활용합니다.

현장 교육의 수업용 점검 규칙은 ‘인원이 10명 이상이면 큰 실습실을 배정한다’입니다. 이상은 경계값 10을 포함합니다. ‘10명 초과’라고 쓰면 11명부터 포함합니다. 말 한 글자의 차이가 비교 기호와 경계 결과를 바꿉니다. 코드를 먼저 외우지 말고 인원 수를 적은 수직선에서 10에 동그라미를 그리고 포함 여부를 표시합니다. 이 규칙은 강의용 공간 배정 예시이며 실제 기관의 정원 기준이 아닙니다.

비교는 질문을 참 또는 거짓으로 답한 값입니다. $9 > 10$ 은 False, $10 > 10$ 은 True, $11 > 10$ 은 True입니다. $>$ 는 초과, $>=$ 는 이상, $<$ 는 미만, $<=$ 는 이하를 나타냅니다. $=$ 는 같은지 묻고 $=$ 는 값에 이름을 붙입니다. 참이라는 결과가 안전하다는 뜻은 아닙니다. 무엇을 비교했는지에 따라 ‘큰 실습실 대상이다’처럼 구체적인 의미를 붙입니다. 참/거짓과 정상/위험은 업무 규칙을 연결해야 설명할 수 있습니다.

그다음 두 갈래 선택을 손으로 추적합니다. 인원 10은 조건이 참이므로 큰 실습실 안내를 선택하고 작은 실습실 안내는 선택하지 않습니다. 인원 9는 조건이 거짓이므로 다른 안내를 선택합니다. 값이 미확인이면 숫자와 비교하기 전에 미확인 갈래로 처리해야 합니다. None을 0으로 바꾸어 작은 방을 정하면 실제 인원 확인이 사라집니다. 최소 검수 표에는 경계 아래 9, 경계 10, 경계 위 11, 미확인 네 사례를 넣습니다. 5와 20만 확인하면 $>$ 와 $>=$ 를 잘못 골라도 둘 다 통과할 수 있습니다. 예상값은 코드보다 먼저 업무 문장으로 정합니다.

새 상황에 독립적으로 적용하기

복습·도전: 대여 정책은 ‘잔여 수량이 5개 이하이면 보충 요청’입니다. 4, 5, 6, 미확인 네 입력의 결과와 비교 기호를 정하세요. 미확인은 보충 확정 대신 확인 요청으로 처리합니다.

정답과 오답의 이유

기호는 $<=$ 이며 4와 5는 보충 요청, 6은 요청 없음, 미확인은 확인 요청입니다. $<$ 를 고르면 경계 5를 놓칩니다. 미확인을 0으로 채우면 수량을 실제 관찰한 것처럼 처리하는 오답입니다.

핵심 세 질문과 해설

1. 이상과 초과 차이 **어느 입력으로 보나요?** 경계값 자체를 넣어 포함 여부를 확인합니다.

2. **True는 늘 정상인가요?** 질문의 의미에 따라 보충·배정·초과 등 다른 업무 뜻이 됩니다.
3. **기대 결과를 실행 뒤 정해도 되나요?** 먼저 업무 문장으로 예상해야 잘못된 결과를 그대로 정답으로 삼지 않습니다.
- 개념.** 비교 연산은 두 값의 관계를 물어 True 또는 False를 만듭니다. >는 초과, >=는 이상, ==는 같은지 확인합니다. =는 대입이므로 역할이 다릅니다.
- 왜 필요한가.** 정책 문구를 코드로 옮길 때 '이상'과 '초과'를 혼동하면 경계값에서 다른 결과가 나옵니다. 기준과 정확히 같은 입력을 확인해야 차이가 보입니다.
- 함께 보는 예.** 온도 24.0과 기준 24.0에서 >는 False, >=는 True입니다. 참·거짓은 비교 질문의 답이며 온도 자체가 참이 나 거짓이라는 의미가 아닙니다.
- 헷갈리는 경우.** '확인 필요'라는 출력은 이 연습의 입력 점검 규칙입니다. 실제 안전 판단이나 센서 성능을 보증하는 말로 사용하지 않습니다.

전체 코드

```
temperature_c = 24.0
limit = 24.0
print(temperature_c > limit)
print(temperature_c >= limit)
print(temperature_c == limit)
```

각 줄의 뜻

줄	코드	뜻
1	temperature_c = 24.0	비교할 측정값을 정합니다.
2	limit = 24.0	수업용 기준을 정합니다.
3	print(temperature_c > limit)	기준을 초과했는지 묻습니다.
4	print(temperature_c >= limit)	기준 이상인지 묻습니다.
5	print(temperature_c == limit)	기준과 같은지 묻습니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A10_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A10_example.py
```

원본 코드의 예상 출력:

```
False
True
True
```

작은 수정: temperature_c = 24.0를 temperature_c = 23.9로 바꿉니다.

수정 후 예상 출력:

```
False
False
False
```

A10 · 따라 하기와 작성칸

A10 · 비교 결과는 참 또는 거짓

목적: 정책 문구를 코드로 옮길 때 '이상'과 '초과'를 혼동하면 경계값에서 다른 결과가 나옵니다. 기준과 정확히 같은 입력을 확인해야 차이가 보입니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A10_input.md](#)
- [A10_example.py](#)
- [A10_notes.md](#)
- [A10_example.py](#)

단계

1. 초과·이상·같은의 문장을 기호와 연결합니다.
2. 기준 아래·같은·위 세 입력에서 True/False를 예상합니다.
3. 한 번에 온도 한 곳만 바꿔 비교 출력 세 줄을 확인합니다.
4. 업무 규칙 '24.0 이상'에 맞는 기호를 고르고 이유를 씁니다.

예상 결과: 같은 값에서 >와 >=의 차이를 설명합니다.

확인 방법: 변수에 값을 넣는 =와 비교하는 ==를 같은 기호처럼 쓰면 될까요?

오류 해결: True/False를 외우기보다 '왼쪽 값이 오른쪽 기준보다 큰가?'라는 질문을 실제 숫자로 다시 읽습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
실제 결과·근거: _____
다음에 확인할 것: _____

A11 · if와 else로 출력 선택하기

개념. if는 조건이 참일 때 실행할 묶음을 선택합니다. else는 그 조건이 거짓일 때 실행할 묶음입니다. 콜론 뒤의 들여쓰 줄들이 그 묶음에 속합니다.

왜 필요한가. 비교 결과를 화면에 보여 주는 단계에서 한 걸음 더 나아가, 비교에 따라 다른 일을 할 수 있습니다. 두 갈래 중 어느 쪽이 실행되는지 먼저 손으로 따라가면 코드를 읽기 쉽습니다.

함께 보는 예. 23.5는 기준 24.0을 넘지 않으므로 else 쪽의 정상이 출력됩니다. 25.0을 넣으면 if 쪽의 확인이 출력됩니다. 두 print를 모두 실행하는 코드와 차이를 비교합니다.

헷갈리는 경우. 들여쓰기를 지우면 문법 오류가 나거나 의도한 묶음이 달라질 수 있습니다. 보기 좋게 정렬하는 장식이 아니라 Python 문장의 구조입니다.

전체 코드

```
temperature_c = 23.5
limit = 24.0
if temperature_c > limit:
    print("확인")
else:
    print("정상")
```

각 줄의 뜻

줄	코드	뜻
1	temperature_c = 23.5	측정값을 정합니다.
2	limit = 24.0	수업용 비교 기준을 정합니다.
3	if temperature_c > limit:	비교가 참인지 확인하고 아래 묶음을 선택합니다.
4	print("확인")	참일 때 확인을 출력합니다.
5	else:	비교가 거짓인 경우의 묶음입니다.
6	print("정상")	거짓일 때 정상을 출력합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A11_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A11_example.py
```

원본 코드의 예상 출력:

정상

작은 수정: 23.5를 25.0로 바꿉니다.

수정 후 예상 출력:

확인

A11 · 따라 하기와 작성칸

A11 · if와 else로 출력 선택하기

목적: 비교 결과를 화면에 보여 주는 단계에서 한 걸음 더 나아가, 비교에 따라 다른 일을 할 수 있습니다. 두 갈래 중 어느 쪽이 실행되는지 먼저 손으로 따라가면 코드를 읽기 쉽습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A11_input.md](#)
- [A11_example.py](#)
- [A11_notes.md](#)
- [A11_example.py](#)

단계

1. 조건식을 동그라미로 표시하고 두 갈래를 말로 설명합니다.
2. 25.0으로 바꿨을 때 실행될 줄만 표시합니다.
3. 들여쓰기를 유지한 채 값 한 곳을 수정해 실행합니다.
4. 수정한 값·실행된 갈래·출력을 노트에 연결합니다.

예상 결과: 23.5는 정상, 25.0은 확인. 양쪽 출력이 동시에 나오지 않습니다.

확인 방법: print("완료")를 들여쓰기 없이 맨 아래에 넣으면 언제 실행될까요?

오류 해결: IndentationError가 나면 콜론 다음 줄의 공백을 원본과 비교합니다. 탭과 공백을 혼용하지 않고 제공 예의 네 칸 공백을 유지합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
 실제 결과·근거: _____
 다음에 확인할 것: _____

A12 · 미측정과 경계값의 세 갈래

개념. elif는 앞 조건이 거짓일 때 다음 조건을 확인합니다. 값이 없다는 상태 None을 먼저 확인하면 숫자가 없는 값을 기준과 비교하는 일을 피할 수 있습니다.

왜 필요한가. 10/8의 미측정을 숫자 0으로 바꾸지 않고 프로그램 안에서도 유지해야 합니다. 미측정·기준 초과·그 밖의 측정을 서로 다른 상태로 구분합니다.

함께 보는 예. None이면 미측정, 숫자 25.0이면 확인, 24.0이면 정상입니다. 기준은 '24.0 초과'이므로 같은 값은 정상 쪽입니다. 숫자 비교는 값이 있는 경우에만 진행됩니다.

헛갈리는 경우. if not temperature_c처럼 쓰면 0.0도 거짓으로 취급될 수 있어 미측정과 섞일 수 있습니다. 오늘은 값 없음의 정확한 비교인 is None을 사용합니다.

전체 코드

```
temperature_c = None

if temperature_c is None:
    print("미측정")

elif temperature_c > 24.0:
    print("확인")

else:
    print("정상")
```

각 줄의 뜻

줄	코드	뜻
1	temperature_c = None	측정값이 없다는 상태를 정합니다.
2	if temperature_c is None:	값 없음인지 먼저 확인합니다.
3	print("미측정")	값이 없으면 미측정이라고 출력합니다.
4	elif temperature_c > 24.0:	값이 있을 때 기준 초과인지 확인합니다.
5	print("확인")	초과하면 확인을 출력합니다.
6	else:	나머지 측정값을 처리합니다.
7	print("정상")	나머지는 정상으로 출력합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.


```
python -X utf8 code/A12_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A12_example.py
```

원본 코드의 예상 출력:

```
미측정
```

작은 수정: temperature_c = None를 temperature_c = 0.0로 바꿉니다.

수정 후 예상 출력:

```
정상
```

A12 · 따라 하기와 작성칸

A12 · 미측정과 경계값의 세 갈래

목적: 10/8의 미측정을 숫자 0으로 바꾸지 않고 프로그램 안에서도 유지해야 합니다. 미측정·기준 초과·그 밖의 측정을 서로 다른 상태로 구분합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A12_input.md](#)
- [A12_example.py](#)
- [A12_notes.md](#)
- [A12_example.py](#)

단계

1. None·0.0·24.0·24.1의 예상 갈래를 표에 씁니다.
2. 원본에서 미측정 확인이 숫자 비교보다 앞선 이유를 설명합니다.
3. work 코드에 하나씩 대입해 출력 또는 강사 실행을 대조합니다.
4. 0.0과 None을 같은 답으로 쓰지 않았는지 확인합니다.

예상 결과: 미측정/정상/정상/확인 순서입니다.

확인 방법: None에 직접 > 24.0을 적용하면 어떻게 될까요?

오류 해결: None을 "None"이라는 문자열로 바꾸지 않습니다. 따옴표가 붙으면 다른 값입니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

5교시 · 여러 기록을 반복하기

15:00~15:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 목록과 반복

먼저 할 활동: A13, A14, A15

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A13 · 리스트와 위치로 여러 값 담기

사례로 이해하기 · 목록의 위치와 반복 누적을 함께 추적하기

필수 설명 보충입니다. 강사 시연 → 함께 추적 → 예상과 실제 비교 → 자기 말로 설명 순서로 읽습니다. 아래 독립 문제는 복습·도전이며 기존 50분 필수 경로와 별도로 활용합니다.

세 배송 상자의 무게가 순서대로 2, 5, 1kg입니다. 목록은 여러 값을 정해진 순서로 묶는 방법입니다. Python의 리스트에서 첫 위치 번호는 0이고 두 번째는 1, 세 번째는 2입니다. 위치 번호는 상자 이름이 아니라 그 목록 안의 순서이므로 정렬하거나 삽입하면 달라질 수 있습니다. 길이 3은 항목 수이며 마지막 위치 번호는 2입니다. 3번째 위치라고 생각해 번호 3으로 읽으면 없는 칸을 찾는 오류가 생깁니다.

각 상자의 무게를 합하려면 반복문으로 한 항목씩 읽고 합계의 현재 값을 갱신합니다. 합계는 반복 전에 0으로 시작합니다. 첫 바퀴는 현재 합계 0에 2를 더해 2, 두 번째는 2에 5를 더해 7, 세 번째는 7에 1을 더해 8입니다. 손 추적표에는 바퀴·현재 항목·변경 전 합계·변경 후 합계를 씁니다. 반복 끝의 총합은 8kg입니다. 반복 안에서 매번 합계를 0으로 다시 만들면 2→5→1처럼 마지막 상자만 남을 수 있습니다.

또 다른 질문은 '4kg 초과인 상자가 몇 개인가'입니다. 첫 2는 조건 거짓이라 건수 0, 다음 5는 참이라 건수 1, 마지막 1은 거짓이라 건수 1입니다. 총무게 8과 초과 상자 수 1은 서로 다른 결과입니다. 누적하는 대상이 무게인지 참인 항목의 개수인지 먼저 정합니다. 반복 본문을 들여쓰는 것은 같은 일을 각 항목에 적용한다는 표시입니다. 완료 안내가 반복 안에 있으면 상자 마다 나오고 밖에 있으면 끝에 한 번 나옵니다. 목록이 비어 있으면 바퀴가 없으므로 합계의 초기값과 완료 안내의 위치를 그대로 추적합니다.

새 상황에 독립적으로 적용하기

복습·도전: 주문별 수량 목록은 3, 0, 7, 2입니다. 총수량과 2개 초과 주문 수를 각각 손 추적표로 구하세요. 길이와 마지막 위치 번호, 빈 목록일 때 두 결과도 적으세요.

정답과 오답의 이유

총수량은 12개이고 초과 주문은 3과 7 두 건입니다. 길이 4, 마지막 번호 3입니다. 빈 목록은 초기 합계 0·건수 0을 유지합니다. 0을 오류라고 임의 삭제하거나 수량 합 12를 주문 건수로 쓰면 서로 다른 질문을 혼동한 것입니다.

핵심 세 질문과 해설

- 1. 첫 위치가 왜 0인가요? Python 리스트가 정한 위치 번호 약속이므로 실제 목록을 그 규칙으로 읽습니다.
 - 2. 합계 초기화는 어디에 놓나요? 이전 바퀴 결과를 유지하도록 반복 전에 한 번 놓습니다.
 - 3. 길이 4이면 번호 4를 읽나요? 마지막 번호는 3이며 번호 4는 범위 밖입니다.
- 개념.** 리스트는 여러 값을 순서대로 묶는 Python 자료구조입니다. 대괄호로 묶고 쉼표로 항목을 나눕니다. 위치 번호는 0부터 시작하며 len은 항목 개수를 돌려줍니다.
- 왜 필요한가.** 측정값이 세 개에서 네 개로 늘어날 때 이름을 계속 만드는 대신 하나의 목록으로 다룰 수 있습니다. 10/8에 읽었던 순서 구조를 직접 작은 값 수정으로 연결합니다.
- 함께 보는 예.** temperatures[0]은 첫 값 22.5이고 len은 3입니다. 마지막 위치 번호는 2이지 3이 아닙니다. 개수와 위치 번호를 서로 다른 질문으로 적습니다.
- 헛갈리는 경우.** [22.5, 24.0]은 두 값의 목록이고 "22.5, 24.0"은 쉼표가 들어 있는 문자열입니다. 곁에 어떤 괄호나 따옴표가 있는지 확인합니다.

전체 코드

```
temperatures = [22.5, 24.0, 23.5]
print(temperatures[0])
print(len(temperatures))
temperatures.append(25.0)
print(len(temperatures))
print(temperatures[3])
```

각 줄의 뜻

줄	코드	뜻
1	temperatures = [22.5, 24.0, 23.5]	온도 세 개를 순서대로 묶습니다.
2	print(temperatures[0])	첫 값을 읽습니다.
3	print(len(temperatures))	현재 개수를 출력합니다.
4	temperatures.append(25.0)	목록 끝에 새 값을 덧붙입니다.
5	print(len(temperatures))	추가 뒤 개수를 확인합니다.
6	print(temperatures[3])	네 번째 값을 출력합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A13_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A13_example.py
```

원본 코드의 예상 출력:

```
22.5
3
4
25.0
```

작은 수정: `append(25.0)`를 `append(26.0)`로 바꿉니다.

수정 후 예상 출력:

```
22.5
3
4
26.0
```

A13 · 따라 하기와 작성칸

A13 · 리스트와 위치로 여러 값 담기

목적: 측정값이 세 개에서 네 개로 늘어날 때 이름을 계속 만드는 대신 하나의 목록으로 다룰 수 있습니다. 10/8에 읽었던 순서 구조를 직접 작은 값 수정으로 연결합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A13_input.md](#)
- [A13_example.py](#)
- [A13_notes.md](#)
- [A13_example.py](#)

단계

1. 추가 전과 후의 목록을 종이에 그립니다.
2. 각 위치 번호와 개수를 따로 표시합니다.

3. 추가할 값을 26.0으로 바꾸고 출력 변화를 예상합니다.

4. 아직 없는 위치를 읽는 오류 사례를 설명합니다.

예상 결과: 추가 후 개수 4, 마지막 값 26.0. 기존 첫 값은 유지됩니다.

확인 방법: 값 하나를 수정하는 것과 append로 추가하는 것은 개수에 어떤 차이가 있나요?

오류 해결: IndexError가 나면 읽는 시점의 목록 길이와 위치 번호를 확인합니다. 뒤에서 추가된다고 앞에서 이미 존재하는 것은 아닙니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과-근거: _____

다음에 확인할 것: _____

A14 · for로 목록을 한 항목씩 읽기

개념. for는 묶음에서 항목을 차례로 꺼내 같은 코드 묶음을 실행합니다. 반복 변수 temperature_c는 이번 차례의 값을 가리키는 이름입니다.

왜 필요한가. 여러 온도에 같은 출력이나 비교를 적용할 때 같은 코드를 복사하지 않아도 됩니다. 항목 수가 늘어나면 반복 횟수도 따라 늘어나는 관계를 관찰합니다.

함께 보는 예. 목록의 첫 22.5가 temperature_c가 되어 출력되고, 다음 차례에는 24.0, 그다음에는 23.5가 됩니다. 반복이 끝난 뒤 들여쓰기 없는 완료가 한 번 출력됩니다.

헛갈리는 경우. 반복 변수에 값을 붙인다고 원래 리스트의 모든 값이 자동 수정되는 것은 아닙니다. 오늘은 목록을 읽는 반복이며 목록 구조를 변경하는 고급 패턴은 다루지 않습니다.

전체 코드

```
temperatures = [22.5, 24.0, 23.5]

for temperature_c in temperatures:
    print(temperature_c)

print("완료")
```

각 줄의 뜻

줄	코드	뜻
1	temperatures = [22.5, 24.0, 23.5]	읽을 값의 순서를 준비합니다.
2	for temperature_c in temperatures:	한 항목씩 현재 값의 이름에 연결합니다.
3	print(temperature_c)	각 차례의 값을 출력합니다.
4	print("완료")	반복이 모두 끝난 뒤 한 번 출력합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A14_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A14_example.py
```

원본 코드의 예상 출력:

```
22.5
24.0
23.5
완료
```

작은 수정: [22.5, 24.0, 23.5]를 [22.5, 24.0, 23.5, 25.0]로 바꿉니다.

수정 후 예상 출력:

```
22.5
24.0
23.5
25.0
완료
```

A14 · 따라 하기와 작성칸

A14 · for로 목록을 한 항목씩 읽기

목적: 여러 온도에 같은 출력이나 비교를 적용할 때 같은 코드를 복사하지 않아도 됩니다. 항목 수가 늘어나면 반복 횟수도 따라 늘어나는 관계를 관찰합니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A14_input.md](#)

- [A14_example.py](#)

- [A14_notes.md](#)

- [A14_example.py](#)

단계

1. 세 차례에 temperature_c가 가리키는 값을 표에 씁니다.

2. 완료가 몇 번 나올지 예상하고 들여쓰기를 근거로 설명합니다.
3. 목록에 25.0을 추가하고 반복 출력 수를 비교합니다.
4. 목록이 비어 있으면 어느 줄이 실행될지 적습니다.

예상 결과: 값은 항목 수만큼 출력되고 완료는 한 번 출력됩니다.

확인 방법: 완료 출력도 네 칸 들여쓰면 무엇이 바뀔까요?

오류 해결: 출력 횟수가 다르면 들여쓰기 범위를 먼저 확인합니다. 코드 줄 수와 실행 횟수는 다를 수 있습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A15 · 반복 안에서 조건 확인과 개수 세기

개념. 반복 안에 조건을 넣으면 각 항목에 같은 판단을 적용할 수 있습니다. 개수를 세는 이름은 반복 전에 0으로 준비하고 조건을 만족할 때 1씩 더합니다.

왜 필요한가. 어떤 기록이 기준을 넘었는지와 전체 몇 건인지 함께 확인할 수 있습니다. 초기화를 반복 안에 넣으면 매번 0으로 돌아가므로 준비 위치가 결과에 영향을 줍니다.

함께 보는 예. 22.5, 24.0, 25.0에서 '24.0 초과'는 마지막 한 건입니다. 24.0은 같은 값이므로 포함하지 않습니다. >를 >=로 바꾸면 두 건이 됩니다.

헛갈리는 경우. count = count + 1은 새 값을 누적하는 대입입니다. 반복 안에서 count = 0을 매번 실행하면 이전 차례의 누적을 잃습니다.

전체 코드

```
temperatures = [22.5, 24.0, 25.0]
count = 0
for temperature_c in temperatures:
    if temperature_c > 24.0:
        count = count + 1
print("확인 건수", count)
```

각 줄의 뜻

줄	코드	뜻
1	<code>temperatures = [22.5, 24.0, 25.0]</code>	점검할 온도 목록을 준비합니다.
2	<code>count = 0</code>	아직 센 기록이 없어서 0으로 시작합니다.
3	<code>for temperature_c in temperatures:</code>	각 온도를 차례로 확인합니다.
4	<code>if temperature_c > 24.0:</code>	이번 온도가 기준을 초과했는지 묻습니다.
5	<code>count = count + 1</code>	조건이 참인 경우만 누적 개수를 하나 늘립니다.
6	<code>print("확인 건수", count)</code>	모든 점검 뒤 최종 개수를 보여 줍니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A15_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A15_example.py
```

원본 코드의 예상 출력:

```
확인 건수 1
```

작은 수정: `> 24.0`를 `>= 24.0`로 바꿉니다.

수정 후 예상 출력:

```
확인 건수 2
```

A15 · 따라 하기와 작성칸

A15 · 반복 안에서 조건 확인과 개수 세기

목적: 어떤 기록이 기준을 넘었는지와 전체 몇 건인지 함께 확인할 수 있습니다. 초기화를 반복 안에 넣으면 매번 0으로 돌아가므로 준비 위치가 결과에 영향을 줍니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A15_input.md](#)

- [A15_example.py](#)

- [A15_notes.md](#)
- [A15_example.py](#)

단계

- 1. 각 차례의 온도·조건 결과·누적 count를 표로 씁니다.
- 2. 초과를 이상으로 바꾸었을 때 영향을 받는 값을 표시합니다.
- 3. 연산자 한 곳을 바꾸고 결과를 대조합니다.
- 4. count의 준비 위치를 반복 앞에 둔 이유를 설명합니다.

예상 결과: 초과는 1건, 이상은 2건이며 경계값 24.0이 차이를 만듭니다.

확인 방법: 한 번의 print인데 세 항목을 검사할 수 있는 이유는 무엇인가요?

오류 해결: 건수가 항상 0 또는 1이면 초기화 위치와 누적 줄의 들여쓰기를 확인합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
실제 결과·근거: _____
다음에 확인할 것: _____

6교시 · 작은 점검 프로그램

16:00~16:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 작은 점검 프로그램

먼저 할 활동: A16, A17, A18

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A16 · 관찰 한 건과 점검표 구조

사례로 이해하기 · 한 기록의 이름표를 지키며 상태표 만들기

필수 설명 보충입니다. 강사 시연 → 함께 추적 → 예상과 실제 비교 → 자기 말로 설명 순서로 읽습니다. 아래 독립 문제는 복습·도전이며 기존 50분 필수 경로와 별도로 활용합니다.

점검 기록 세 건에 장치명과 온도가 함께 있습니다. 순서대로 D01은 18°C, D02는 미확인, D03은 26°C입니다. 온도만 따로 출력하면 어느 장치의 값인지 읽습니다. 한 관찰을 이름표가 붙은 값 묶음, 즉 딕셔너리로 만들고 여러 관찰을 리스트로 모읍니다. 바깥 목록은 기록들을 차례로 묶고 안쪽 이름표는 같은 기록의 항목을 구별합니다. 위치 번호와 항목 이름은 찾는 방식이 다르므로 두 겹 구조를 분리해서 봅니다.

완성 원문은 D01/18, D02/None, D03/26 세 기록입니다. 수업용 기준은 '온도가 25°C 초과이면 확인, 그 외 숫자는 정상, 미확인은 미측정'입니다. 첫 기록에서 장치명 D01과 숫자 18을 꺼내고 None인지 먼저 묻습니다. 아니므로 18>25를 확인

해 정상으로 표시합니다. 둘째는 None이라 미측정으로 표시하고 숫자 비교를 하지 않습니다. 셋째는 26>25가 참이므로 확인입니다. 최종 세 줄은 D01 정상, D02 미측정, D03 확인입니다.

이름표 temperature_c와 temperature는 비슷한 뜻이어도 다른 키입니다. 없는 이름표를 직접 찾으면 KeyError가 발생할 수 있습니다. 오류가 없어도 임의로 새 키를 만들기 전에 원자료와 약속한 헤더를 비교합니다. 같은 장치가 다른 시각에 다시 측정되면 기록이 하나 더 생길 수 있으므로 장치명만으로 중복 삭제하지 않습니다. 오늘의 상태 프로그램은 제공 숫자를 기준과 비교할 뿐 실제 센서 고장이나 안전을 확정하지 않습니다. 새 입력을 넘길 때 기록의 출처·기준·미확인 처리·예상 결과를 함께 적어 다음 사람이 같은 판단을 재현할 수 있게 합니다.

새 상황에 독립적으로 적용하기

복습·도전: 세 물품 P01 수량 0, P02 수량 미확인, P03 수량 2를 대상으로 '2개 이하이면 주문 요청' 상태표를 새로 설계하세요. 물품번호와 수량의 이름표, 미확인 갈래를 먼저 정하고 세 줄을 작성하세요.

정답과 오답의 이유

이름표는 예를 들어 product_id와 quantity입니다. P01-P03은 주문 요청, P02는 수량 확인입니다. 0은 실제 수량이라 포함하고 None은 비교 전에 처리합니다. temperature_c 키를 그대로 쓰는 답은 새 업무의 항목 뜻을 맞추지 못한 것입니다.

핵심 세 질문과 해설

1. **목록과 딕셔너리가 함께 필요한 이유는?** 여러 건의 순서와 한 건 안의 항목 이름을 각각 보존하기 위해서입니다.
2. **미확인을 숫자 비교 뒤에 두나요?** 숫자로 비교할 수 없으므로 먼저 분기합니다.
3. **상태표는 실제 고장을 증명하나요?** 제공 입력과 수업 규칙을 적용한 결과이며 실제 원인은 별도 확인합니다.

개념. 딕셔너리는 이름표인 키와 값의 묶음입니다. 여러 관찰 딕셔너리를 리스트로 묶으면 '기록 여러 건'의 구조가 됩니다. 이 개념은 10/8의 표 행과 JSON 객체를 프로그램 안에 옮긴 것입니다.

왜 필요한가. 온도만 나열하면 어느 센서인지 놓치기 쉽습니다. 식별자와 온도를 한 기록으로 묶어 처리하면 출력의 대상을 분명히 할 수 있습니다. 오늘은 제공된 구조를 읽고 한 값만 바꿉니다.

함께 보는 예. records는 기록 두 건의 목록입니다. 첫 기록의 sensor_id는 S01이고 temperature_c는 22.5입니다. for의 record는 이번 차례의 딕셔너리 한 개를 가리킵니다.

헛갈리는 경우. record[0]처럼 위치로 읽는 것과 record["sensor_id"]처럼 키로 읽는 것을 섞지 않습니다. 이 딕셔너리에는 숫자 0이라는 키를 준비하지 않았습니다.

전체 코드

```
records = [
    {"sensor_id": "S01", "temperature_c": 22.5},
    {"sensor_id": "S02", "temperature_c": 24.0},
]
for record in records:
    print(record["sensor_id"], record["temperature_c"])
```

각 줄의 뜻

줄	코드	뜻
1	<code>records = [</code>	기록을 담을 리스트를 시작합니다.
2	<code>{"sensor_id": "S01", "temperature_c": 22.5},</code>	첫 관찰을 두 키의 딕셔너리로 넣습니다.
3	<code>{"sensor_id": "S02", "temperature_c": 24.0},</code>	둘째 관찰을 넣습니다.
4	<code>]</code>	리스트를 닫습니다.
5	<code>for record in records:</code>	딕셔너리 한 개씩 읽습니다.
6	<code>print(record["sensor_id"], record["temperature_c"])</code>	이번 관찰의 이름표로 두 값을 찾아 출력합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A16_example.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A16_example.py
```

원본 코드의 예상 출력:

```
S01 22.5
S02 24.0
```

작은 수정: `"temperature_c": 24.0`를 `"temperature_c": 25.0`로 바꿉니다.

수정 후 예상 출력:

```
S01 22.5
S02 25.0
```

A16 · 따라 하기와 작성칸

A16 · 관찰 한 건과 점검표 구조

목적: 온도만 나열하면 어느 센서인지 놓치기 쉽습니다. 식별자와 온도를 한 기록으로 묶어 처리하면 출력의 대상을 분명히 할 수 있습니다. 오늘은 제공된 구조를 읽고 한 값만 바꿉니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A16_input.md](#)
- [A16_example.py](#)
- [A16_notes.md](#)
- [A16_example.py](#)

단계

1. 바깥 리스트와 안쪽 딕셔너리를 표시합니다.
2. S02의 온도를 바꾸었을 때 어느 출력만 바뀌는지 예상합니다.
3. 값 한 곳을 수정해 출력과 관찰 대상이 유지되는지 확인합니다.
4. 표의 한 행과 record의 관계를 문장으로 설명합니다.

예상 결과: S02의 온도만 25.0으로 바뀝니다. 기록 수는 두 건입니다.

확인 방법: 딕셔너리 키 temperature_c를 오타로 읽으면 무엇부터 확인하나요?

오류 해결: 괄호 짝을 찾기 어렵다면 먼저 리스트 외곽, 다음 각 딕셔너리 외곽을 따로 표시합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
 실제 결과·근거: _____
 다음에 확인할 것: _____

A17 · 수업용 센서 요약 완성 예제

- 개념.** 종합 예제는 목록·반복·조건을 결합하여 각 관찰의 상태를 출력합니다. 새 문법을 많이 더하기보다 이미 읽은 조각이 어디에 놓였는지 연결합니다.
- 왜 필요한가.** 실무에서는 한 줄의 문법을 아는 것과 여러 단계가 연결된 프로그램을 읽는 것이 다릅니다. 준비한 입력, 각 차례의 처리, 최종 출력을 나누어 표시하면 긴 코드도 추적할 수 있습니다.
- 함께 보는 예.** S01의 22.5는 정상, S02의 25.0은 확인, S03의 None은 미측정입니다. 상태 문자열을 status라는 이름에 넣은 뒤 공통 print 한 곳에서 센서명과 함께 출력합니다.
- 헛갈리는 경우.** 상태 표시만으로 실제 장비의 안전을 판정하지 않습니다. 24.0이라는 숫자는 수업 비교 규칙입니다. 상태가 None인지 먼저 검사하므로 미측정을 실제 0으로 채우지 않습니다.

전체 코드

```
records = [
    {"sensor_id": "S01", "temperature_c": 22.5},
    {"sensor_id": "S02", "temperature_c": 25.0},
    {"sensor_id": "S03", "temperature_c": None},
]

limit = 24.0

for record in records:
    temperature_c = record["temperature_c"]
    if temperature_c is None:
        status = "미측정"
    elif temperature_c > limit:
        status = "확인"
    else:
        status = "정상"
    print(record["sensor_id"], status)
```

각 줄의 뜻

줄	코드	뜻
1	<code>records = [</code>	관찰 목록을 시작합니다.
2	<code>{"sensor_id": "S01", "temperature_c": 22.5},</code>	첫 측정 기록입니다.
3	<code>{"sensor_id": "S02", "temperature_c": 25.0},</code>	둘째 측정 기록입니다.
4	<code>{"sensor_id": "S03", "temperature_c": None},</code>	온도가 없는 셋째 기록입니다.
5	<code>]</code>	목록을 닫습니다.
6	<code>limit = 24.0</code>	수업용 기준을 정합니다.
7	<code>for record in records:</code>	기록 한 건씩 처리합니다.
8	<code>temperature_c = record["temperature_c"]</code>	이번 기록의 온도를 읽습니다.
9	<code>if temperature_c is None:</code>	값 없음부터 확인합니다.
10	<code>status = "미측정"</code>	값이 없을 때의 상태를 정합니다.
11	<code>elif temperature_c > limit:</code>	값이 있으면 초과 여부를 묻습니다.
12	<code>status = "확인"</code>	초과한 경우의 상태를 정합니다.
13	<code>else:</code>	앞의 두 경우가 아닐 때의 묶음입니다.
14	<code>status = "정상"</code>	나머지 측정의 상태를 정합니다.
15	<code>print(record["sensor_id"], status)</code>	이번 센서와 판정 상태를 출력합니다.

실행과 예상 출력

현재 위치가 압축을 푼 course 폴더일 때 실행합니다.

```
python -X utf8 code/A17_sensor_summary.py
```

수정한 작업본은 다음 명령으로 실행합니다.

```
python -X utf8 work/A17_sensor_summary.py
```

원본 코드의 예상 출력:

```
S01 정상
S02 확인
S03 미측정
```

작은 수정: limit = 24.0를 limit = 25.0로 바꿉니다.

수정 후 예상 출력:

```
S01 정상
S02 정상
S03 미측정
```

A17 · 따라 하기와 작성칸

A17 · 수업용 센서 요약 완성 예제

목적: 실무에서는 한 줄의 문법을 아는 것과 여러 단계가 연결된 프로그램을 읽는 것이 다릅니다. 준비한 입력, 각 차례의 처리, 최종 출력을 나누어 표시하면 긴 코드도 추적할 수 있습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A17_input.md](#)
- [A17_sensor_summary.py](#)
- [A17_notes.md](#)
- [A17_sensor_summary.py](#)

단계

1. 각 기록의 입력·조건·예상 상태를 작성합니다.
2. limit만 25.0으로 바꾸고 달라질 기록을 고릅니다.
3. 전체 파일을 저장·실행하고 세 출력 줄을 대조합니다.
4. 미측정 처리와 기준 비교의 실행 순서를 짝에게 설명합니다.

예상 결과: 기준을 25.0으로 바꾸면 S02는 같은 값이므로 정상이며 S03은 계속 미측정입니다.

확인 방법: 기준만 바꿨는데 미측정 상태가 달라진다면 무엇을 점검해야 할까요?

오류 해결: 한꺼번에 여러 곳을 수정하지 않습니다. 원본과 작업본을 나란히 보고 지정한 기준 한 곳부터 다시 비교합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____
 실제 결과·근거: _____
 다음에 확인할 것: _____

A18 · 새 입력 검수와 실행 인계

- 개념.** 프로그램 사용 결과는 파일·입력·명령·출력·판단 근거를 함께 남길 때 다른 사람이 확인할 수 있습니다. 잘 실행되었다는 한 문장보다 어떤 사례를 확인했는지가 중요합니다.
- 왜 필요한가.** 새 입력에서 경계값과 미측정을 다시 확인하면 예제 답을 외운 것인지 원리를 이해했는지 볼 수 있습니다. 실행하지 못한 경우에는 예상·강사 관찰·개인 실행을 정확히 구별합니다.
- 함께 보는 예.** 새 입력은 S04의 24.0, S05의 0.0, S06의 None입니다. 기준은 24.0 초과입니다. 정상·정상·미측정으로 예상하고, 기존 A17 작업본의 기록 목록만 바꾸는 수정안을 작성합니다.
- 헛갈리는 경우.** 과거 A17 출력 화면을 새 입력의 결과라고 제출하면 재현 근거가 맞지 않습니다. 파일을 저장한 시점과 실행한 경로, 새 입력값을 함께 확인해야 합니다.

제공 입력

새 관찰: S04=24.0, S05=0.0, S06=None. 기준 24.0 초과.

인계 양식: 실행 파일 / 바꾼 입력 / 실행 명령 / 예상 출력 / 실제 확인 방식 / 남은 질문.

A18 · 따라 하기와 작성칸

A18 · 새 입력 검수와 실행 인계

- 추가 원문·작업 파일:
- [A18_input.md](#)
 - [A18_notes.md](#)
 - [A18_final.py](#)
- 목적:** 새 입력에서 경계값과 미측정을 다시 확인하면 예제 답을 외운 것인지 원리를 이해했는지 볼 수 있습니다. 실행하지 못한 경우에는 예상·강사 관찰·개인 실행을 정확히 구별합니다.
- 준비:** 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.
- [A18_input.md](#)
 - [A18_notes.md](#)

단계

1. 새 관찰 세 개의 예상 상태와 근거를 적습니다.
2. A17 구조를 재사용해 work/A18_final.py의 기록 목록을 새 자료로 바꿉니다.
3. 같은 기준으로 실행하거나 제공 출력과 수동 대조하고 확인 방식을 기록합니다.

4. 짝이 같은 파일을 찾고 결과를 설명할 수 있게 인계 세 문장을 씁니다.

예상 결과: S04 정상, S05 정상, S06 미측정. 0.0과 None이 다르게 처리됩니다.

확인 방법: 내일 파일에서 읽은 "24.0"을 그대로 이 코드에 넣으면 무엇이 추가로 필요할까요?

오류 해결: 새 파일이 실행되지 않으면 먼저 원본 A17을 실행해 실행 환경과 새 수정의 문제를 나눕니다. 원본까지 실패하면 환경 단서부터 확인합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

추가 활동 · 필수 경로를 마친 뒤

아래는 기존 웹의 개념·활동입니다. 수업 시간의 필수 결과를 먼저 확인한 뒤 선택하거나 복습에 사용합니다. 기존 웹의 활동 별 분량 배정은 이번 PDF의 50분 교시 시간표와 별도로 보존된 설계입니다.

A03 · 저장·실행 순서와 오류 단서

개념. 편집기의 변경 내용은 저장하기 전까지 파일에 반영되지 않을 수 있습니다. 실행 명령은 디스크에 저장된 특정 파일을 읽습니다. 터미널은 명령을 입력하고 실행 결과를 보는 창이며 코드 편집기와 역할이 다릅니다.

왜 필요한가. 같은 이름의 원본과 작업본이 있을 때 다른 파일을 실행하면 수정한 문구가 나오지 않습니다. 재실행을 반복하기 전에 파일 경로와 저장 여부를 확인하면 원인을 좁힐 수 있습니다.

함께 보는 예. 작업본에는 새 문구가 보이지만 code/A02_print.py를 실행했다면 원본 문구가 나오는 것이 자연스럽습니다. 명령 끝의 경로를 work/A02_print.py로 맞춰야 작업본을 실행합니다.

헛갈리는 경우. SyntaxError는 Python이 코드를 문장으로 읽지 못했다는 단서입니다. 화면 전체를 지우거나 Python을 재 설치하기 전에 마지막 오류 종류와 해당 줄, 바로 앞 줄의 따옴표·괄호를 확인합니다.

제공 입력

```

사례 1: 편집기 탭에는 work/A02_print.py, 터미널 명령은 python -X utf8 code/A02_print.py.
사례 2: print("점검 시작") → SyntaxError.
사례 3: print("점검 시작")를 고쳤지만 저장 표시가 남아 있음.
    
```

A03 · 저장·실행 순서와 오류 단서

목적: 같은 이름의 원본과 작업본이 있을 때 다른 파일을 실행하면 수정한 문구가 나오지 않습니다. 재실행을 반복하기 전에 파일 경로와 저장 여부를 확인하면 원인을 좁힐 수 있습니다.

준비: 원자료와 작업 파일을 나란히 엽니다. 원본은 보존하고 work 파일에 기록합니다. 실행 환경이 없으면 같은 입력·코드·출력으로 수동 추적합니다.

- [A03_input.md](#)

- [A03_notes.md](#)

단계

1. 세 사례에서 보이는 사실과 추정 원인을 다른 칸에 적습니다.
2. 각 사례의 다음 확인 동작을 하나만 고릅니다.
3. 잘못된 따옴표 코드를 종이에 고친 뒤 괄호와 따옴표의 짝을 표시합니다.
4. 작업본 저장 후 동일 경로를 실행하는 순서를 네 단계로 작성합니다.

예상 결과: 다른 파일 실행·닫는 따옴표 누락·미저장 가능성을 각각 구별합니다.

확인 방법: 오류 줄 번호가 마지막 줄을 가리키면 그 줄만 보면 충분할까요?

오류 해결: 오류 메시지를 해석하기 어렵다면 명령, 실행 파일 경로, 오류 마지막 줄을 그대로 기록해 강사에게 보여 줍니다. 비슷한 문구로 요약해 단서를 잃지 않습니다.

용어와 코드 기호 찾아보기

Python 기초 쉬운 용어

용어	뜻
코드	컴퓨터가 할 일을 정해진 문법으로 적은 내용
실행	저장한 코드의 명령을 컴퓨터가 수행하는 과정
변수	프로그램 안에서 값에 붙인 이름
문자열	글자의 순서를 값으로 다루는 자료형
조건문	참·거짓 결과에 따라 실행할 부분을 선택하는 문장
반복문	목록의 항목마다 같은 부분을 실행하는 문장
들여쓰기	같은 작업 안에 속한 줄을 공백으로 묶는 표기
None	현재 값이 없음을 나타내는 Python 값

용어 이름만 외우기보다 활동의 값·파일·출력에서 무엇을 가리키는지 짚습니다. 같은 숫자 모양이라도 의미와 저장 자료형을 구별합니다.

오늘 배운 내용 확인하기

Python 기초 확인과 평가

18활동의 작성·예상·실제 확인을 분리해 봅니다. 속도나 설치 성공만 평가하지 않습니다.

기준	확인할 증거	보완이 필요한 경우
개념	입력·처리·출력 또는 변경 단계의 역할 설명	결과 이름만 외우고 근거를 말하지 못함
수정	지정한 위치와 변경 전후를 연결	여러 곳을 바꾸어 원인을 설명하지 못함
확인	정상·경계·오류 사례의 예상과 실제 구별	실행 성공을 내용 정확성으로 단정
전달	파일·명령·결과·남은 질문을 재현 가능하게 기록	어느 파일을 사용했는지 불명확

각 기준은 '독립 설명 가능 / 힌트 후 가능 / 다시 연습'으로 기록합니다. 개인 실행 여부는 별도 체크하며 미실행을 실패한 개념 이해로 간주하지 않습니다.

구간별 확인 질문

- 1. 오류 줄 번호가 마지막 줄을 가리키면 그 줄만 보면 충분할까요?
- 2. 오류가 사라졌다는 것만으로 올바른 수정이라고 할 수 있나요?
- 3. 코드가 종료 코드 0으로 끝났다면 평균의 업무 의미도 보장되나요?
- 4. None에 직접 > 24.0을 적용하면 어떻게 될까요?
- 5. 한 번의 print인데 세 항목을 검사할 수 있는 이유는 무엇인가요?
- 6. 내일 파일에서 읽은 "24.0"을 그대로 이 코드에 넣으면 무엇이 추가로 필요할까요?

제출

work의 활동 노트와 수정 파일, 마지막 종합 작업, 확인한 명령·출력 또는 수동 추적 기록을 제출합니다. 제출 위치는 강사 안내를 따릅니다. 정답 폴더를 학생 결과에 복사하지 않습니다.

2026-10-05 보강 · 새 사례 전이 문제

기존 평가를 보존합니다. 아래 여섯 문제는 복습·도전용이며 설치 성공과 개념 이해를 구별해 평가합니다. 답을 먼저 읽지 말고 근거와 예상값을 기록합니다.

문제 1 · 편집 화면·저장 파일·실행 결과의 세 상태

복습·도전: 편집기는 work/check.py에 '검사 B'를 보여 줍니다. 실행 명령은 code/check.py를 가리키고 출력은 '검사 A'입니다. 확인된 사실 세 가지와 다음 확인 동작 두 가지를 적으세요. 새 설치나 임의 파일 삭제는 필요하지 않습니다.

정답·해설: 편집 경로·실행 경로가 다르며 화면 출력은 A입니다. work 파일을 저장했는지 확인하고 같은 work 주소를 실행합니다. 파일이 다르다는 증거는 있지만 저장 여부는 아직 미확인입니다. 'Python 고장'은 증거 없는 원인 단정입니다.

문제 2 · 재고 갱신을 한 줄씩 추적하기

복습·도전: 물 12병을 준비하고 4병을 배부한 뒤 현재 수량을 안내합니다. 다시 6병을 받아 최종 수량을 안내합니다. 두 안내 값과 세 번의 대입 뒤 현재 수량을 표로 작성하고, 두 print를 어디에 놓을지 판단하세요.

정답·해설: 대입 뒤 현재 값은 12→8→14이고 안내값은 8과 14입니다. 배부 직후와 추가 입고 직후에 출력해야 합니다. 모든 대입 뒤에 같은 이름을 두 번 출력하면 14·14가 되어 중간 안내를 잃습니다.

문제 3 · 운송비 계산에서 문자열과 숫자 나누기

복습·도전: 판매표의 주문번호 "0009", 단가 "1500", 수량 "3"을 보고 총액과 각 항목의 저장 계획을 쓰세요. 단가는 정수원, 수량은 개수라는 계약입니다. 배송비 2000원을 포함한 최종액도 계산하세요.

정답·해설: 주문번호는 문자열 유지, 단가와 수량은 정수로 해석합니다. 상품 총액 1500×3=4500원, 최종액 6500원입니다. "1500"+"3"의 문자열 연결은 "15003"으로 금액 계산이 아닙니다. 주문번호를 정수 9로 바꾸면 앞 0을 잃습니다.

문제 4 · 이상과 초과를 업무 문장으로 읽기

복습·도전: 대여 정책은 '잔여 수량이 5개 이하이면 보충 요청'입니다. 4, 5, 6, 미확인 네 입력의 결과와 비교 기호를 정하세요. 미확인은 보충 확정 대신 확인 요청으로 처리합니다.

정답·해설: 기호는 ≤이며 4와 5는 보충 요청, 6은 요청 없음, 미확인은 확인 요청입니다. <를 고르면 경계 5를 놓칩니다. 미확인을 0으로 채우면 수량을 실제 관찰한 것처럼 처리하는 오답입니다.

문제 5 · 목록의 위치와 반복 누적을 함께 추적하기

복습·도전: 주문별 수량 목록은 3, 0, 7, 2입니다. 총수량과 2개 초과 주문 수를 각각 손 추적표로 구하세요. 길이와 마지막 위치 번호, 빈 목록일 때 두 결과도 적으세요.

정답·해설: 총수량은 12개이고 초과 주문은 3과 7 두 건입니다. 길이 4, 마지막 번호 3입니다. 빈 목록은 초기 합계 0·건수 0을 유지합니다. 0을 오류라고 임의 삭제하거나 수량 합 12를 주문 건수로 쓰면 서로 다른 질문을 혼동한 것입니다.

문제 6 · 한 기록의 이름표를 지키며 상태표 만들기

복습·도전: 세 물품 P01 수량 0, P02 수량 미확인, P03 수량 2를 대상으로 '2개 이하이면 주문 요청' 상태표를 새로 설계하세요. 물품번호와 수량의 이름표, 미확인 갈래를 먼저 정하고 세 줄을 작성하세요.

정답·해설: 이름표는 예를 들어 product_id와 quantity입니다. P01·P03은 주문 요청, P02는 수량 확인입니다. 0은 실제 수량이라 포함하고 None은 비교 전에 처리합니다. temperature_c 키를 그대로 쓰는 답은 새 업무의 항목 뜻을 맞추지 못한 것입니다.

파일과 다음 수업

링크를 눌러 파일을 내려받습니다. 예제 ZIP은 압축을 풀고 README부터 읽습니다. 원본 code·data는 보존하고 work의 작업본을 수정합니다. 파일이 저장된 폴더와 확장명을 확인하세요.

실습 예제 코드와 데이터

Python 기초 PPT

처음 시작 안내

용어와 코드 기호

학생용 교안

학생 실습 안내

확인 문제

학생 교안 PDF · 내용 보강본

설치·로그인·장비 준비와 학습 성취는 구분합니다. 이 교안의 배정 시간은 실제 초심자 리허설 결과가 아닙니다. 코드·제공 출력·실물 검증의 한계는 해당 웹 수업과 교안 설명을 따릅니다.